

SUPPLEMENT

RapidMiner AI Studio 補充教材

Supplement — Getting Started with RapidMiner AI Studio

這份補充教材回答四個問題：為什麼這門課用它、怎麼免費取得、它有哪些功能與演算法、以及它跟 Python 到底是什麼關係。

本補充教材大綱

01 ★ 核心論點 · 流程與 Python 幾乎 1:1

02 為什麼這門課用它 · 三個理由

03 它是什麼、誰做的 · RapidMiner → Altair → Siemens

04 如何取得免費授權 · 三條路徑

05 安裝與啟用 · 常見卡關點

06 介面六大區塊 · 先熟悉哪兩塊

07 四個核心概念 · Process · Operator · Port · ExampleSet

08 你的第一個流程 · 六個運算子跑完一個專案

09 ★ 運算子 ↔ Python 對照 · 逐步詳解

10 內建演算法清單 · 依四大家族整理

11 擴充套件生態 · 本課程必裝的四個

12 三大生產力工具 · Turbo Prep · Auto Model · Deployments

13 Execute Python · 兩個世界的橋樑

14 什麼時候該改用 Python · 不是二選一

15 學習資源與認證 · 免費課程與證照

16 快捷鍵與錯誤排除 · 新手九成的錯誤

17 常見問題與限制 · 誠實說明

PART 01

核心論點

Why This Tool Transfers

先講結論：你在 AI Studio 學到的東西，換到 Python 一步都不會少。
這是這門課選擇它的最主要理由。

★ AI Studio 的專案流程與 Python 幾乎 1:1 對應



九個步驟一步不差

讀取 → 清理 → 特徵工程 → 設目標欄 → 切分 → 訓練 → 套用 → 評估 → 交叉驗證。

★ Set Role = y = df['label']

指定哪一欄是要預測的目標。
概念完全相同，只是一個用拖拉、一個用程式碼。

★ Apply Model

= model.predict()。把訓練好的模型套到新資料上。

★ Cross Validation = cross_val_score()

AI Studio 把它做成一個可放進其他運算子的巢狀容器——這反而比程式碼更看得見結構。

運算子 ↔ Python 對照 (一) 資料處理

任務	AI Studio 運算子	Python (pandas / scikit-learn)
讀取資料	Retrieve / Read CSV / Read Excel	pd.read_csv() / pd.read_excel()
查看統計	Statistics (結果頁)	df.describe() / df.info()
篩選列	Filter Examples	df[df['age'] > 65]
選擇欄	Select Attributes	df[['age','bmi','sex']]
新增計算欄	Generate Attributes	df['bmi'] = df['w'] / df['h']**2
缺值處理	Replace Missing Values	df.fillna() / SimpleImputer()

★ 這幾張對照表建議印出來放在旁邊。學會左邊那一欄，右邊那一欄你只需要查語法。

運算子 ↔ Python 對照（二）特徵與切分

任務	AI Studio 運算子	Python （ pandas / scikit-learn ）
類別轉數值	Nominal to Numerical	pd.get_dummies() / OneHotEncoder()
標準化	Normalize	StandardScaler()
★ 指定目標欄	★ Set Role （ label ）	★ y = df['label']
合併資料	Join	pd.merge()
群組彙總	Aggregate	df.groupby().agg()
切分訓練測試	Split Data	train_test_split()

★ 第三列是 AI Studio 特有的：Set Role 把「哪一欄是 y」變成欄位的屬性，而不是另外開一個變數。

運算子 ↔ Python 對照 (三) 建模

任務	AI Studio 運算子	Python
決策樹	Decision Tree	DecisionTreeClassifier()
隨機森林	Random Forest	RandomForestClassifier()
樸素貝氏	Naive Bayes	GaussianNB() / MultinomialNB()
k 近鄰	k-NN	KNeighborsClassifier()
線性迴歸	Linear Regression	LinearRegression()
邏輯斯迴歸	Logistic Regression (H2O)	LogisticRegression()
k-Means 分群	k-Means	KMeans()
主成分分析	PCA	PCA()

★ 名字幾乎一樣 —— 因為兩邊用的是同一套演算法，只是包裝不同。

運算子 ↔ Python 對照 (四) 套用與評估

任務	AI Studio 運算子	Python
★ 套用模型	★ Apply Model	★ model.predict()
評估分類	Performance (Classification)	classification_report()
評估迴歸	Performance (Regression)	mean_squared_error() / r2_score()
★ 交叉驗證	★ Cross Validation (子流程)	★ cross_val_score()
參數最佳化	Optimize Parameters (Grid)	GridSearchCV()
特徵重要性	Weight by ... / 模型輸出	model.feature_importances_

★ 注意交叉驗證：AI Studio 把它做成一個「容器」，訓練與測試流程放在它裡面。這個巢狀結構比 cross_val_score() 更看得見在做什麼。

為什麼這門課用 AI Studio

① 把注意力留給「方法」

不是留給語法

- 初學者常卡在：縮排錯了、套件裝不起來、版本不相容

★ 這些跟資料科學一點

關係都沒有

- AI Studio 把這一層拿掉了

卡住的是語法，不是方法

② 流程視覺化＝思路視覺化

看得見資料怎麼流

- Python 的流程藏在程式碼的先後順序裡
- AI Studio 的流程就畫在畫布上

★ 學生看得見「資料現在在哪」

- 除錯時每個連線點都能看

中間結果隨時看得到

③ 概念可完整遷移到 Python

不是死路一條

★ 九個步驟與 Python 一一對應

- 學會 Set Role ,
就懂 `y = df['label']`
- 學會 Cross Validation 子流程,
就懂 `cross_val_score()`

轉 Python 只需補語法

① 把注意力留給「方法」

初學者常卡在縮排、套件安裝、版本不相容

★ 這些跟資料科學一點關係都沒有。

② 流程視覺化＝思路視覺化

Python 的流程藏在程式碼順序裡；
AI Studio 的流程就畫在畫布上，資料現在在哪一眼就看得得到。

★ ③ 概念可完整遷移

不是死路一條。未來要轉 Python，
只需要學語法，不需要重學概念。

課程立場

AI Studio 不是「Python 的替代品」，是「學概念的鷹架」。

★ 這門課的立場：AI Studio 不是「Python 的替代品」，是「學概念的鷹架」。

護理與健康科技背景的你，要學的是「怎麼判斷」，而不是「怎麼寫迴圈」。

★ 等概念穩了，語法隨時可以補——反過來先學語法的人，常常寫得出程式卻說不出為什麼。

順帶回答一個常見疑問：那我到底要不要學 Python？

要，但不是現在，也不是為了這門課

- ▶ ★ 這門課的目標是讓你「會判斷」，不是「會寫程式」
- ▶ 先把九個步驟的概念弄懂，之後學 Python 會快非常多
- ▶ 反過來先學語法再學概念的人，常常寫得出程式但說不出為什麼

什麼時候該開始學 Python

- ▶ ★ 你已經能獨立完成一個完整專案（本課程結束時應該可以）
- ▶ 你遇到 AI Studio 做不到的事——通常是很後面的事
- ▶ 你要寫論文、要用最新的方法、要跟工程團隊協作

學的時候怎麼接軌

- ▶ ★ 拿本補充教材的對照表，一一列列把語法補上去
- ▶ 先用 Execute Python 在熟悉的流程裡插一小段（PART 06）
- ▶ M10 的 Vibe Coding 也是一條路：請 AI 幫你把流程翻成程式碼

※ 課程不禁止用 Python 交作業。但若你選擇用 Python，請確保你說得出每一步在做什麼——那才是評分的重點。

PART 02

它是什麼、從哪來

What It Is & Where It Came From

名字換過好幾次，這是很多人搞混的原因。

名稱沿革：為什麼你會看到不同的名字

時期	名稱	說明
2001-2007	YALE	德國多特蒙德工業大學的研究專案（Yet Another Learning Environment）
2007-2022	RapidMiner Studio	★ 最廣為人知的名字。大量教材、書籍、影片仍用這個名稱
2022-2024	RapidMiner（Altair 收購）	Altair 併購 RapidMiner
2024-2025	Altair AI Studio	Altair 時期的名稱。市面上多數教材、影片停在這個名字
2025-	★ RapidMiner AI Studio	Siemens 併購 Altair 後，官方文件已改回這個名稱。最新版 2026.1.1（2026/04）

★ 實務上的意義：搜尋資料時用「RapidMiner」關鍵字往往找到更多內容；副檔名 .rmp 也還是 RapidMiner 的縮寫。安裝檔、授權頁與社群仍大量並用兩個名字，本講義一律簡稱「AI Studio」。

它在資料科學工具中的定位

同類型的工具

- ▶ KNIME —— 最接近的競品，也是視覺化流程
- ▶ Orange —— 教學導向，更輕量
- ▶ SAS Enterprise Miner —— 商業老牌，昂貴
- ▶ Azure ML Designer / SageMaker Canvas —— 雲端版
- ▶ ★ 共同點：都是「視覺化流程 + 低程式碼」

AI Studio 的相對優勢

- ▶ ★ 免費版功能完整（多數競品免費版有明顯限制）
- ▶ 運算子數量多、文件完整（每個都有說明與範例）
- ▶ ★ 教育資源豐富：免費線上課程 + 認證
- ▶ 全球逾 4,000 所大學使用，教材與案例多
- ▶ 可與 Python / R 混用，不是封閉系統

PART 03

如何取得免費授權

Getting a Free License

完全免費，而且沒有功能閹割。

但有兩套授權系統並存，選錯會卡關——這一節說清楚。

三條取得授權的路徑

取得免費授權（完全免費）

路徑 A 學生版

Altair Student Edition

- ① 搜尋 Altair Student Edition 頁面
- ② 用學校 email 註冊帳號
- ③ 完成 email 驗證與學籍填寫
- ④ 以 Altair One 帳號啟用

- ★ 12 個月、可續約至畢業
- ★ 無功能限制
- ★ 本課程建議走這條

路徑 B 個人版

RapidMiner Personal Edition

- ① 到 my.rapidminer.com 註冊
- ② 免費取得 Educational 授權
- ③ 下載 AI Studio 安裝
- ④ 於軟體內輸入授權

- ★ 一年期、可續約
- 不含互動分析；過期降為 1 萬列版
- 走 RapidMiner 那套系統

路徑 C 教師

Academic Hub

- ① 到 Academic Hub 教師頁面
- ② 申請課程用授權
- ③ 取得 8 模組 Course Kit
- ④ 可申請課堂批次授權

- 全球逾 4,000 所大學使用
- 含現成教材與認證
- 可申請課堂批次授權

★ 本課程建議走「路徑 A 學生版」：用學校 email 註冊、無功能限制、可續約到畢業。

※ 兩套授權系統（Altair One 與 RapidMiner）並存中，啟用出問題多半是選錯授權類型。

★ 顯示「找不到授權」時：Help → Manage Licenses → 手動貼上授權碼。

※ 用私人信箱註冊會被導向付費頁面 —— 一定要用學校 email。

★ 路徑 A 學生版

用學校 email 註冊 Altair 學生方案。
12 個月、可續約至畢業、無功能限制。★ 本課程建議走這條。

路徑 B 個人版

到 my.rapidminer.com 註冊，取得免費的 Educational / Personal 授權。一年可續；不含互動分析，過期未續會降為 1 萬列上限的免費版。

路徑 C 教師

Academic Hub 提供課程用授權，另有 8 模組現成 Course Kit 可下載。

★ 常見卡關原因

兩套授權系統（Altair One 與 RapidMiner）並存中。
啟用出問題，多半是選錯授權類型。

申請與安裝：逐步操作（一）註冊與安裝

★ 建議在第一次上課前先完成，課堂不會有時間等大家安裝

安裝指引（一）

【步驟一】註冊帳號

1. 到 Altair 學生方案頁面（搜尋「Altair Student Edition」）
2. ★ 用學校 email（@ntunhs.edu.tw）註冊
—— 用私人信箱會被導向付費頁面
3. 完成 email 驗證與學籍資訊填寫

【步驟二】下載安裝檔

1. 登入後在下載區找 RapidMiner AI Studio（舊頁面可能仍寫 Altair AI Studio）
2. 選對作業系統（Windows / macOS / Linux）
3. ★ 檔案約 1–2 GB，請預留時間與磁碟空間
（安裝完成後約佔 3–4 GB）

【步驟三】安裝與啟用

1. 執行安裝檔，安裝路徑用預設值
★ 路徑含中文或空白字元可能導致啟動失敗
2. 第一次啟動會要求登入，用剛才註冊的 Altair One 帳號
3. 若顯示「找不到授權」：
Help → Manage Licenses → 手動貼上授權碼

【硬體建議】

- RAM 8 GB 起跳，16 GB 較舒適
- 有獨立顯卡跑深度學習會快很多，但非必要

申請與安裝：逐步操作（二）擴充套件與驗證

★ 這一步最常被漏掉，也是課程範例打不開的頭號原因

安裝指引（二）

【步驟四】★ 安裝本課程需要的四個擴充套件

操作路徑：Extensions → Marketplace，搜尋名稱後點 Install

- Text Processing → M6 文字探勘（斷詞、TF-IDF、n-gram）
- Deep Learning → M5 深度學習（DL4J 各層、CNN、LSTM）
- H2O → M5 表格資料的深度學習與 GBM
- Generative Models → M7 生成式 AI（Hugging Face 模型）

★ 裝完必須重新啟動軟體才會生效

★ 全部免費，不需要額外授權

【步驟五】驗證安裝是否成功

1. 開啟課程範例中任一個 .rmp 檔
2. 按 Ctrl+Enter 執行
3. 若能跑完並跳到 Results 頁 → 安裝正確

★ 若運算子上出現紅色驚嘆號，訊息寫「缺少運算子」或
「operator not found」→ 就是擴充套件沒裝齊，回到步驟四

【建議的第一個練習】

開啟 M0 的範例流程，對著每一條連線點按滑鼠中鍵，
看看資料在每一步長什麼樣 —— 這是最快建立直覺的方法。

安裝與啟用的六個常見問題

症狀	可能原因	解法
註冊時被導向付費頁面	用了私人 email	★ 一定要用學校 email 註冊
啟動後顯示「授權過期」	一年期授權到期	重新登入帳號續約；學生身分可續到畢業
找不到授權 / 啟用失敗	兩套授權系統選錯	Help → Manage Licenses 手動貼授權碼
開啟範例檔出現紅色驚嘆號	★ 擴充套件沒裝	Extensions → Marketplace 補裝，重開軟體
軟體啟動很慢或當掉	記憶體不足	關掉其他程式；或在設定中調整 JVM 記憶體上限
中文路徑導致讀檔失敗	路徑含中文或空白	★ 把資料放到純英文路徑（如 D:\data\）

★ 第四項最常見：課程範例大量使用 Text Processing 與 Deep Learning 運算子，沒裝就開不起來。

PART 04

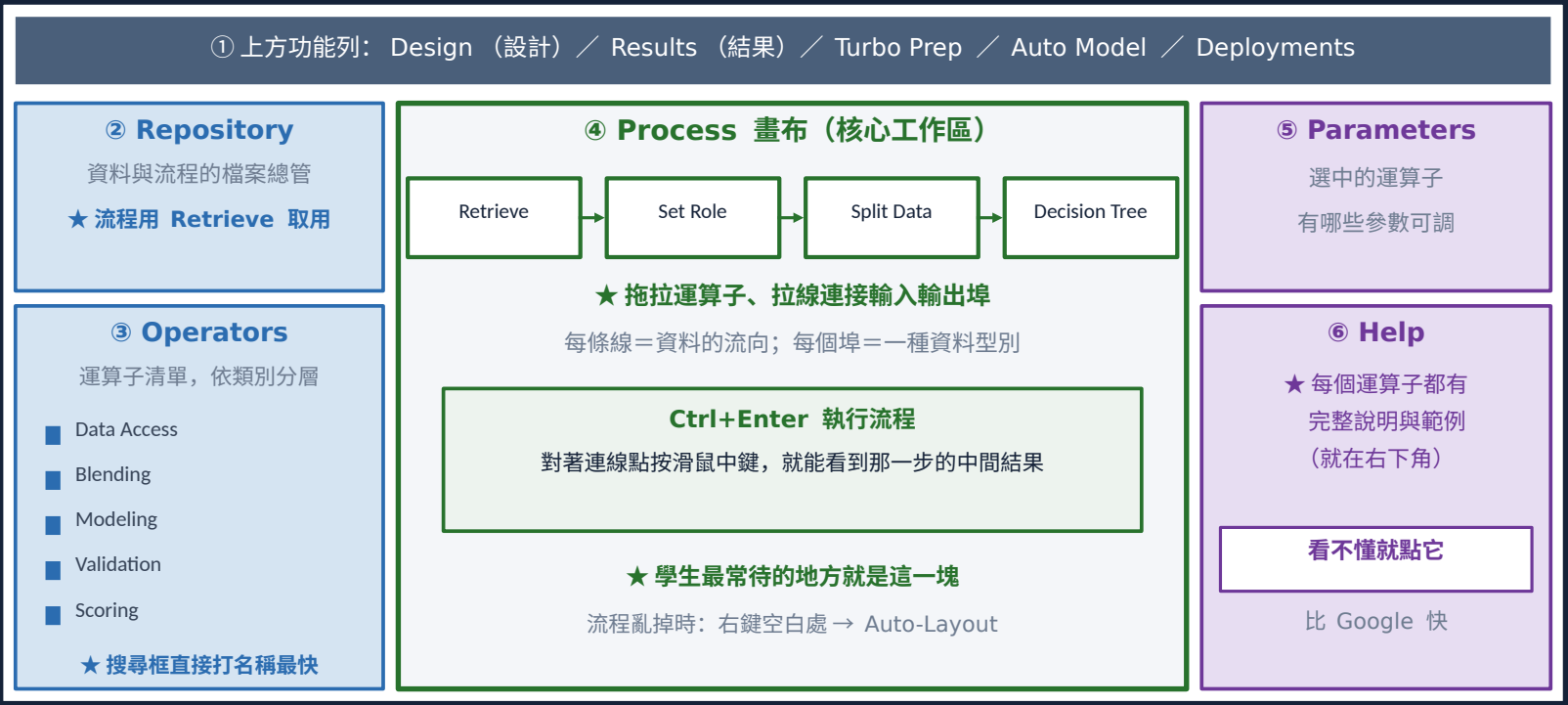
介面與核心概念

Interface & Core Concepts

四個概念弄懂，你就能看懂任何一個 .rmp 流程檔。

介面的六個區塊

軟體畫面的區塊配置示意（不是截圖）：哪個面板在哪、資料怎麼流



★ 給新手的一句話：先熟悉「④ 畫布」與「⑥ Help」這兩塊，其他都能慢慢摸。

六個區塊裡，你每天真正會用到的只有兩個：中間的畫布，與右下角的 Help。

※ 這是區塊配置示意圖，不是真實截圖 —— 版本不同，面板位置可能略有差異。

④ Process 畫布

核心工作區。拖拉運算子、拉線連接輸入輸出埠。

★ ⑥ Help

每個運算子都有完整說明與範例，就在右下角。看不懂就點它——比 Google 快。

③ Operators

運算子清單，依 Data Access / Blending / Modeling / Validation / Scoring 分層。

★ 給新手的一句話

先熟悉「④ 畫布」與「⑥ Help」這兩塊，其他都能慢慢摸。

四個核心概念

概念	是什麼	對應到 Python 的什麼
Process 流程	整個分析流程，存成一個 .rmp 檔	一個 .py 檔或 .ipynb notebook
Operator 運算子	流程中的一個處理步驟（一個方塊）	一個函式呼叫，如 <code>pd.read_csv()</code> 或 <code>model.fit()</code>
Port 埠	運算子的輸入與輸出接點；連線就是資料的流向	★ 函式的參數與回傳值。連線＝變數傳遞
ExampleSet 資料集	AI Studio 的資料表格型別	★ pandas DataFrame。列＝Example、欄＝Attribute

★ 特別注意 ExampleSet 的用語：一列叫 Example（不是 row）、一欄叫 Attribute（不是 column）。看文件時要習慣這套詞彙。

ExampleSet： AI Studio 的資料表格

角色	id	regular	regular	regular	★ label
型別	整數	numerical	nominal	numerical	binominal
欄名	id	age	sex	bmi	readmit
第 1 列	1	72	男	24.8	yes
第 2 列	2	65	女	31.2	no
第 3 列	3	81	男	22.1	yes

↑ 一列叫 **Example** (不是 row)

↑ 一欄叫 **Attribute** (不是 column)

對應到 pandas

ExampleSet $\approx$ DataFrame

Example $\approx$ row

Attribute $\approx$ column

★ 但多了「**Role**」這個概念：

pandas 沒有內建的方式

標記「哪一欄是 y」

AI Studio 把它做成欄位屬性

五種常見的 Role

regular

一般特徵欄 (預設)

★ label

要預測的目標欄

id

識別碼，不參與建模

weight

樣本權重

prediction / confidence

模型輸出時自動加上

★ 忘記 **Set Role** 是初學者最常見的錯誤：建模運算子會直接報錯說「找不到 label」。

★ 用語要習慣：一列叫 **Example** (不是 row)、一欄叫 **Attribute** (不是 column)。

ExampleSet $\approx$ pandas DataFrame，但多了「Role」—— pandas 沒有內建方式標記「哪一欄是 y」。

★ 忘了 **Set Role** 是最常見的錯誤；型別不對是第二常見。

用語要習慣

一列叫 **Example** (不是 row)、一欄叫 **Attribute** (不是 column)。
看官方文件時會一直遇到。

★ 多了 Role 這個概念

pandas 沒有內建方式標記「哪一欄是 y」；

AI Studio 把它做成欄位的屬性。

五種常見 Role

regular、★ label、id、weight、prediction / confidence。

★ 最常見的錯誤

忘記 **Set Role** → 建模運算子直接報錯說「找不到 label」。

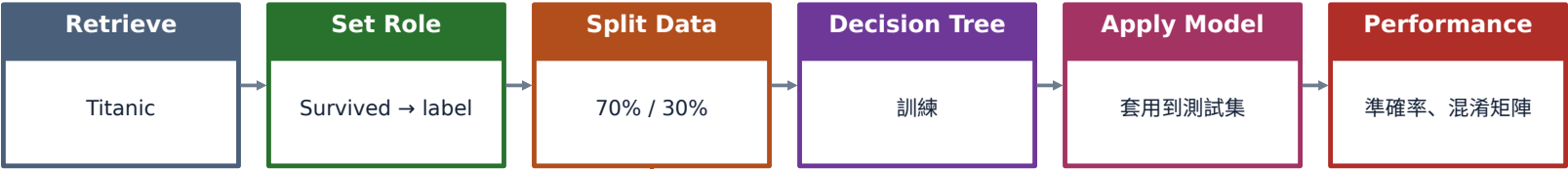
三種常見的檔案格式

格式	是什麼	怎麼處理
.rmp	★ 流程檔。本質是 XML，記錄有哪些運算子、怎麼連接、參數設定	可用文字編輯器開啟檢視；可放進 Git 但 diff 難讀
.rmhdf5table	AI Studio 的資料檔（HDF5 格式），存在 Repository 裡	★ 只能用 AI Studio 開。要給別人請另存 CSV
Repository	不是單一檔案，是一個受管理的資料夾結構	★ 流程中用 Retrieve 取用；比直接讀路徑穩定（換電腦不會壞）

★ 實務建議：資料放 Repository、流程用 Retrieve 取用。直接寫死絕對路徑的流程，換一台電腦就跑不動。

你的第一個流程：六個運算子

★ 六個運算子，一個完整的機器學習專案



③ 的「測試」那條線，要接到 ⑤ Apply Model 的第二個輸入

- ① **Retrieve** 從 Repository 拉出資料。也可用 Read CSV 讀外部檔
- ② **Set Role** ★ 最容易忘的一步。指定 **Survived** 為 **label**
- ③ **Split Data** 拉出兩條線：上面接訓練、下面接測試
- ④ **Decision Tree** 輸入訓練資料，輸出一個「模型」（不是預測值）
- ⑤ **Apply Model** ★ 兩個輸入：模型 + 測試資料。輸出才是預測值
- ⑥ **Performance** 輸入帶預測的資料，輸出評估指標

★ 第 ⑤ 步最容易接錯：Apply Model 需要「兩個」輸入 —— 模型從 ④ 來、資料從 ③ 來。

④ Decision Tree 輸出的是「模型」不是預測值 —— 預測值要等 Apply Model 之後才出現。

★ 建議：第一次上課就自己拉一遍這六個。拉完你就懂了一半。

六個運算子跑完一個完整專案

Retrieve → Set Role → Split Data → Decision Tree → Apply Model → Performance。

★ 第 ⑤ 步最容易接錯

Apply Model 需要「兩個」輸入 —— 模型從 Decision Tree 來、資料從 Split Data 的測試那條線來。

④ 輸出的是「模型」不是預測值

很多人在這裡搞混。
預測值要等 Apply Model 之後才出現。

★ 建議

第一次上課就自己拉一遍這六個。拉完你就懂了一半。

三個一定要弄懂的機制

① Role（角色）：告訴系統這一欄是幹嘛的

- ▶ regular —— 一般特徵欄（預設）
- ▶ ★ label —— 要預測的目標欄（= Python 的 y）
- ▶ id —— 識別碼，不參與建模
- ▶ prediction / confidence —— 模型輸出時自動加上的欄位

② 子流程（Subprocess）：運算子裡面還有運算子

- ▶ Cross Validation、Optimize Parameters、Loop 都是容器
- ▶ ★ 雙擊它會進入內層畫布，裡面再放訓練與測試的流程
- ▶ 這個設計比 Python 更看得見結構——巢狀關係是畫出來的

③ 資料型別會影響能用哪些運算子

- ▶ nominal（類別）、numerical（數值）、date、text 各有適用的運算子
- ▶ ★ 很多「運算子灰掉不能連」的問題，都是型別不對
- ▶ 常用轉換：Nominal to Numerical、Numerical to Polynomial、Nominal to Text

PART 05

支援的演算法與功能

Algorithms & Capabilities

內建演算法逾 60 種，另可透過擴充套件加上更多。

以下依家族整理，並標出本課程實際會用到的。

內建演算法的四大家族

★ 標星號者為本課程實際使用（依 277 個範例流程檔統計）

內建演算法逾 60 種

監督式一分類

- ★ **Decision Tree**
- ★ **Random Forest**
- ★ **Naive Bayes**
- ★ **k-NN**
- Logistic Regression
- SVM (LibSVM)
- Rule Induction
- Gradient Boosted Trees
- Neural Net

監督式一迴歸

- ★ **Linear Regression**

Polynomial Regression

Generalized Linear Model

Regression Tree

SVM Regression

Deep Learning (H2O)

kNN Regression

非監督式

- ★ **k-Means**
- ★ **X-Means**

DBSCAN

Agglomerative

Support Vector Clustering

- ★ **PCA / SVD**

Self-Organizing Map

其他任務

- ★ **FP-Growth**（關聯規則）
- ★ **Association Rules**
- ★ **LOF**（離群偵測）

One-Class SVM

Anomaly Detection

時間序列（擴充套件）

★ 標星號者為本課程實際使用（依 277 個範例流程檔的運算子統計）。

整體內建演算法逾 60 種，另可透過 Weka、H2O 與其他擴充套件再加上更多。

★ 名字幾乎與 **scikit-learn** 一樣 —— 因為兩邊用的是同一套演算法，只是包裝不同。

監督式一分類

決策樹、隨機森林、樸素貝氏、k-NN、邏輯斯迴歸、SVM、規則歸納、梯度提升樹、神經網路。

監督式一迴歸

線性迴歸、多項式迴歸、廣義線性模型、迴歸樹、SVM 迴歸、深度學習。

非監督式

k-Means、X-Means、DBSCAN、階層式分群、PCA / SVD、SOM。

★ 其他任務

FP-Growth 關聯規則、LOF 離群偵測、異常偵測、時間序列（擴充）。

本課程各模組實際用到的運算子（上）

模組	主要運算子	週次
M1 資料工程	Replace Missing Values、Normalize、Detect Outlier Nominal to Numerical、Generate Attributes	W2-W3
M2-1 迴歸	Linear Regression、Polynomial Regression Performance (Regression)	W4
M2-2 分類	★ Decision Tree、Naive Bayes、k-NN、Random Forest Compare ROCs、Optimize Threshold	W5
M3 非監督式	k-Means、X-Means、PCA、SVD Cluster Model Visualizer	W6

★ 想複習某個演算法時，直接到對應模組的範例資料夾找 .rmp 檔，開起來看流程怎麼組。

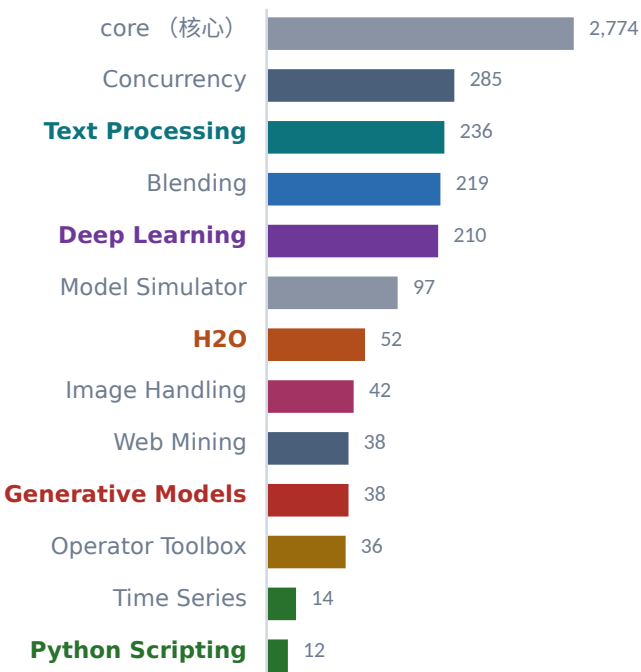
本課程各模組實際用到的運算子（下）

模組	主要運算子	週次
M4 進階監督式	Bagging、AdaBoost、Vote、Stacking ★ FP-Growth、LOF、Anomaly Detection	W7
M5 深度學習	★ DL4J 各層（Dense / Conv / Pooling / LSTM / Dropout） H2O Deep Learning、Import Zoo Model	W8-W9
M6 文字探勘	★ Process Documents、Tokenize、Filter Stopwords Generate n-Grams、TF-IDF	W11
M7 生成式 AI	★ Generative Models：Hugging Face 模型、Text2Text Finetune、Embeddings	W12

★ 依 277 個範例流程檔的實際統計。M5 的 DL4J 相關運算子出現 210 次，是課程中最密集使用的擴充套件之一。

擴充套件生態：功能靠 Extension 加上去

本課程 277 個流程檔的實際使用次數（對數刻度）



課程用到的關鍵擴充套件

Text Processing
斷詞、停用詞、TF-IDF、n-gram → M6 文字探勘

Deep Learning (DL4J)
逐層堆疊、CNN、LSTM、遷移學習 → M5 深度學習

H2O
另一套深度學習與 GBM 實作 → M5 表格資料主力

Generative Models
Hugging Face 模型、微調、嵌入 → M7 生成式 AI

Python Scripting
★ 在流程中插入 Python 程式碼 → 兩個世界的橋樑

左圖是實際使用統計

本課程 277 個流程檔中，各擴充套件出現的次數。

★ 必裝的四個

Text Processing、Deep Learning、H2O、Generative Models。沒裝就開不了課程範例檔。

安裝方式

Extensions → Marketplace，搜尋後點 Install，重開軟體。全部免費。

★ Python Scripting

在流程中插入 Python 程式碼——兩個世界的橋樑，見 PART 06。

★ 必裝四個擴充：Text Processing、Deep Learning、H2O、Generative Models

安裝方式：Extensions → Marketplace，搜尋後點 Install，★ 裝完必須重開軟體才會生效。全部免費。

★ 沒裝這四個就開不了課程範例檔——運算子會出現紅色驚嘆號，訊息寫「operator not found」。

值得知道的其他擴充套件

擴充套件	做什麼	什麼時候會用到
Text Processing	斷詞、停用詞、詞幹、TF-IDF、n-gram	★ M6；任何文字資料
Deep Learning (DL4J)	逐層堆疊網路、CNN、LSTM、遷移學習	★ M5 影像任務
H2O	另一套 ML 實作：深度學習、GBM、GLM	★ M5 表格資料
Generative Models	Hugging Face 模型、微調、嵌入、向量檢索	★ M7 生成式 AI
Operator Toolbox	補足核心缺少的實用運算子	流程寫到一半發現「怎麼沒有這個」時
Time Series	時間序列的滯後特徵、分解、預測	M4 時間序列延伸
Web Mining	抓網頁、解析 HTML	要自己蒐集網路文字資料時
Image Handling	讀取影像、抽取影像特徵	M5 影像任務的前處理
Python / R Scripting	在流程中執行 Python 或 R	需要客製邏輯時
Weka	接上 Weka 的演算法庫	想試核心沒有的演算法時

★ Marketplace 上有上百個擴充套件，多數免費。但別亂裝——裝太多會拖慢啟動速度，且運算子清單會變得很難找。

除了畫布之外的三大工具

Turbo Prep

互動式資料準備

- 像 Excel 一樣直接看資料改資料
- 清理、轉換、合併都有精靈引導
- 改完可一鍵轉成 Process

★ 適合：探索期快速試

但改完要轉成 Process
否則無法重現

Auto Model

自動建模

- 自動試多種演算法並比較
- 自動處理前處理與參數
- 產出模型比較表與解釋圖

★ 適合：快速建立基準線

★ 別直接交作業
它不會告訴你為什麼

Deployments

模型部署

- 把訓練好的模型發佈成服務
- 可設定定期執行
- 產生 REST API 供外部呼叫

★ 適合：專題要 Demo 時

免費版有功能限制
完整功能需 AI Hub

Turbo Prep

像 Excel 一樣互動式地清理資料，有精靈引導。

★ 但改完要轉成 Process，否則無法重現。

Auto Model

自動試多種演算法並比較，產出模型比較表。

★ 適合快速建立基準線。

Deployments

把模型發佈成服務、設定排程、產生 REST API。
免費版有功能限制。

★ 課程立場：Auto Model 可以用來建立基準線，但作業與專題必須自己拉流程。

因為這門課要學的是「為什麼這樣做」，不是「怎麼按到結果」。

★ Turbo Prep 改完的資料記得一鍵轉成 Process —— 否則換一台電腦就重現不了。

★ 課程立場

Auto Model 可用來建立基準線，但作業與專題必須自己拉流程——
因為要學的是「為什麼這樣做」。

PART 06

與 Python 的關係

Interoperability with Python

它不是封閉系統。

你可以在流程中間插一段 Python，也可以把流程整個換成 Python。

Execute Python：兩個世界的橋樑

在熟悉的流程中間插一段 Python：上游自動轉 DataFrame，回傳後再轉回 ExampleSet



★ 什麼時候該用

- 非常特殊的計算邏輯
- 要呼叫特定 Python 套件
- 複雜的字串處理
- 現成運算子做不到的事

Execute Python 裡面長這樣

```
def rm_main(data):  
    # data = 上游 ExampleSet，已自動轉成 DataFrame  
    data['bmi'] = data['w'] / data['h']**2  
    return data          # 回傳後再轉回 ExampleSet  
    回傳的 DataFrame 會自動轉回 ExampleSet 接給下游
```

★ 什麼時候不該用

- 有現成運算子就別寫
- 寫了就失去「流程看得見」的好處
- 也失去可讀性

什麼時候該插一段 Python

★ 有現成的運算子就別寫 Python —— 寫了就失去「流程看得見」的好處

- 值得寫的情況：非常特殊的計算邏輯、要呼叫特定的 Python 套件、複雜的字串處理
- 這也是「從 AI Studio 過渡到 Python」的好起點：先在熟悉的流程裡插一小段

★ ExampleSet ⇌ DataFrame 的轉換，Execute Python 幫你做掉了。

先在熟悉的流程裡插一小段 Python，比一開始就整個換成程式碼溫和得多。

怎麼運作

上游的 ExampleSet 會自動轉成 pandas DataFrame，你寫一個 `rm_main(data)` 函式處理它，回傳後再轉回 ExampleSet。

★ 什麼時候該用

非常特殊的計算邏輯、要呼叫特定 Python 套件、複雜的字串處理。

★ 什麼時候不該用

有現成運算子就別寫——寫了就失去「流程看得見」的好處。

★ 過渡的好起點

先在熟悉的流程裡插一小段 Python，比一開始就整個換成程式碼溫和得多。

AI Studio 還是 Python：怎麼選

★ AI Studio 比較合適

視覺化流程・低程式碼

- 學習與教學：概念看得見
- 標準流程專案：讀檔→清理→建模→評估
- 需要快速試多種演算法比較
- 團隊裡有非工程背景的人要一起看

★ 流程要交給別人接手維護

★ Python 比較合適

程式碼・可版控

- 要用最新的演算法或論文實作

★ 大規模資料與分散式運算

- 要做客製化的深度學習架構
- 要整合進既有的軟體系統

★ 需要版本控制與自動化測試（CI/CD）

★ 這不是二選一：.rmp 流程檔可以匯出，Execute Python 也能混用。

實務常見做法：用 AI Studio 快速探索與建立基準線，確定方向後再用 Python 做正式實作與部署。

★ 對本課程學生最重要的一句話

你在這門課學到的九個步驟，換到 Python 一步都不會少。

工具會換，流程不會 —— 那才是真正帶得走的東西。

AI Studio 比較合適

學習與教學、標準流程專案、快速比較多種演算法、

★ 流程要交給別人接手維護。

Python 比較合適

最新演算法、大規模與分散式運算、客製深度學習架構、

★ 需要版本控制與 CI/CD。

★ 不是二選一

實務常見做法：用 AI Studio 快速探索與建立基準線，確定方向後再用 Python 做正式實作與部署。

★ 對你最重要的一句話

你在這門課學到的九個步驟，換到 Python 一步都不會少。
工具會換，流程不會。

值得記住的快捷鍵與操作技巧

操作	方式	為什麼有用
執行流程	Ctrl + Enter (或 F11)	★ 最常用
看中間結果	★ 對著任一條連線點按滑鼠中鍵	不用改流程就能看資料在這一步長什麼樣
快速找運算子	在 Operators 搜尋框直接打名稱	比在分類樹裡翻快得多
加入運算子	把運算子拖到「現有連線」上，會自動插入	不用手動重接線
暫時停用某個運算子	右鍵 → Disable Operator	除錯時比刪掉再加回來方便
加註解	右鍵 → Add Comment	★ 交作業與給別人看時很重要
整理版面	右鍵空白處 → Auto-Layout / Rearrange	流程亂掉時一鍵整齊
看運算子說明	選中它，看右下角 Help 面板	★ 比 Google 快
匯出流程	File → Export Process	交作業用；含所有參數設定

★ 第二項（滑鼠中鍵看中間結果）是最被低估的功能。它等同於 Python 的 `print(df.head())`，但不用改任何東西。

常見錯誤訊息與排除

錯誤訊息	意思	解法
Input ExampleSet has no label attribute	沒有指定目標欄	★ 加一個 Set Role ，把目標欄設成 label
Operator cannot handle nominal attributes	運算子吃不下類別型資料	加 Nominal to Numerical 轉換
Missing values are not allowed	資料還有缺值	加 Replace Missing Values 或 Filter Examples
Operator not found / 紅色驚嘆號	★ 擴充套件沒裝	Extensions → Marketplace 補裝，重開軟體
Cannot connect ports (type mismatch)	埠的資料型別不合	★ 檢查上下游：模型接模型埠、資料接資料埠
Java heap space / OutOfMemory	記憶體不足	先抽樣（Sample）；或調高 JVM 記憶體上限
Repository entry not found	流程找不到資料	資料被搬走或改名；重新指定 Retrieve 的來源

★ 前三個佔了新手九成的錯誤，而且都出在同一個原因：資料還沒準備好就接到建模運算子。

給這門課學生的實用建議（上）：交付與可讀性

① 流程要加註解，且檔名要有意義

- ▶ ★ 作業評分會看流程的可讀性，不是只看結果
- ▶ 檔名建議：M4_集成比較_v2.rmp，不要叫 process(3).rmp
- ▶ 關鍵步驟加 Comment 說明「為什麼這樣做」

② 資料放 Repository，不要寫死絕對路徑

- ▶ ★ 用 Retrieve 取用，換電腦或交給老師才不會壞掉
- ▶ 要交檔案時：Export Process + 附上資料，或說明資料來源
- ▶ 路徑含中文或空白也常出問題，資料夾建議用純英文

這兩點直接影響作業分數：老師打不開你的流程，就沒辦法給分。

給這門課學生的實用建議（下）：除錯與進修

③ 善用滑鼠中鍵看中間結果

- ▶ ★ 每加一個運算子就看一次，比全部接完才發現錯了好得多
- ▶ 這跟 Python 裡「每寫幾行就 print 一次」是同一個習慣

④ 不確定就看 Help 面板

- ▶ 每個運算子都有完整說明、參數解釋與範例流程
- ▶ ★ 右下角，選中運算子就會顯示 —— 比搜尋網路快也準

⑤ 學期中考一張認證

- ▶ RapidMiner Academy 的課程免費，Professional 是第一級
- ▶ ★ 通過可得數位徽章，可放進履歷與 LinkedIn
- ▶ 投入時間不多，但對非資訊背景的你是很好的佐證

★ 最後補一句：遇到錯誤先看訊息本身。AI Studio 的錯誤訊息寫得比多數 Python 套件清楚得多。

PART 07

學習資源與限制

Resources & Honest Limitations

免費資源比你想要的多。

但它也有做不到的事——這一節誠實說明。

官方免費學習資源

資源	內容	建議用法
Academic Hub	學生／教師專屬入口、Course Kit（8 模組）	★ 申請授權的起點
RapidMiner Academy	微模組線上課程：應用案例、機器學習、資料工程、部署管理	★ 每個約 10-20 分鐘，適合課後補強
認證考試	Professional（第一級）、Master（第二級）分不同知識領域，可獨立取得	★ 通過可獲數位徽章，可放進履歷與 LinkedIn
Goldmine	完整記錄的真實案例：零售定價、儀表數位化、工單分類、產品推薦	看別人怎麼組流程，比看教學更有感
Community 論壇	官方社群，可提問	★ 遇到錯誤訊息先搜這裡
內建 Help 面板	每個運算子的完整說明與範例	★ 最快的求助方式，就在右下角

★ 課程建議：學期中至少完成 RapidMiner Academy 的三門微課程，並嘗試考一張 Professional 認證——對履歷有實質幫助。

誠實說明：它做不到或不擅長的事（上）

① 最新的演算法會慢一拍

- ▶ 論文發表的新方法，通常要等一段時間才有對應的運算子
- ▶ ★ Python 生態（PyTorch、Hugging Face）永遠更快
- ▶ 折衷：用 Execute Python 或 Generative Models 擴充補上

② 大規模資料會吃力

- ▶ ★ 單機工具，資料量到千萬列以上會明顯變慢
- ▶ 沒有原生的分散式運算（那要 AI Hub 或其他平台）
- ▶ 折衷：先抽樣建模，確定方向後再用其他工具跑全量

這兩點是「能力邊界」，多數課堂與研究專案不會踩到。

誠實說明：它做不到或不擅長的事（下）

③ 版本控制與協作較弱

- ▶ ★ .rmp 是 XML 檔，可以放進 Git，但 diff 幾乎不可讀
- ▶ 多人同時改同一個流程很容易衝突
- ▶ 折衷：拆成多個小流程；重要版本另存檔名加日期

④ 產業採用度不如 Python

- ▶ ★ 職缺要求多半寫 Python / R，很少寫 RapidMiner
- ▶ 但這不影響它作為學習工具的價值 —— 概念是共通的
- ▶ 把它當成「學概念的鷹架」，而不是「最終要用的工具」

★ 這一頁刻意放在最後：任何工具都有限制，說得出限制才表示你真的了解它。這也呼應 M9 的立場。

這份補充教材的重點

- 1 ★ 核心論點： AI Studio 的九個步驟與 Python 幾乎 1:1 對應
- 2 `Set Role = y = df['label'] ; Apply Model = model.predict()`
- 3 Cross Validation 子流程 = `cross_val_score()`，而且結構更看得見
- 4 免費取得：★ 用學校 email 走學生版，12 個月可續約至畢業
- 5 ★ 開課前務必裝好四個擴充套件： Text Processing、 Deep Learning、 H2O、 Generative Models
- 6 四個核心概念： Process、 Operator、 Port、 ExampleSet